

Solutions for Exercise Set 1

Note that the model answers contain more code than is required from the actual answers. The code is included for your reference (for the full source code, see the corresponding R markdown file). Some questions in the problem sheet are marked “optional”: you should not reduce points if an answer to such a question is missing.

While we have tried to make the answers comprehensive and correct it is possible that there are mistakes and omissions; please tell us if you find anything!

Problem 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Task a

```
df = pd.read_csv("./npf_train.csv")
df = df.drop(columns=["id", "partlybad"], axis=1)
df.head(10)
```

```
##          date      class4  C02168.mean  ...  UV_B.std   CS.mean   CS.std
## 0  2000-03-21          II    372.396757  ...   0.104295  0.000510  0.000123
## 1  2000-03-23    nonevent    372.889867  ...   0.229672  0.000706  0.000250
## 2  2000-04-07          Ia    373.869464  ...   0.451130  0.000851  0.000244
## 3  2000-04-09          Ib    376.006588  ...   0.572409  0.002083  0.000203
## 4  2000-04-14    nonevent    374.068239  ...   0.106017  0.002650  0.000891
## 5  2000-04-19    nonevent    376.229945  ...   0.608225  0.006348  0.002127
## 6  2000-04-23          II    375.426310  ...   0.476821  0.003680  0.002160
## 7  2000-05-28          II    373.885289  ...   0.962612  0.003949  0.001230
## 8  2000-06-01          II    370.104178  ...   0.646785  0.002491  0.000343
## 9  2000-06-03    nonevent    372.547940  ...   0.424066  0.001714  0.000870
##
```

```
## [10 rows x 102 columns]
```

```
# R
df = read.csv("./npf_train.csv")
df = df[, -c(1,4)]
# View(df)
```

Task b

```
df[["T84.mean", "UV_A.mean", "CS.mean"]].describe()
```

```
##          T84.mean  UV_A.mean   CS.mean
## count    450.000000  450.000000  450.000000
## mean       6.439594   10.797330    0.003083
## std        9.827282    6.626678    0.002246
```

```
## min      -23.627710      0.438965      0.000343
## 25%       -0.733160       4.304971      0.001436
## 50%        7.661622      11.506937      0.002548
## 75%       14.118346      16.573188      0.004119
## max       27.001804      22.500037      0.013706
```

```
# R
summary(df[,c("T84.mean", "UV_A.mean", "CS.mean")])
```

```
##      T84.mean      UV_A.mean      CS.mean
## Min.      :-23.6277 Min.      : 0.439 Min.      :0.0003425
## 1st Qu.: -0.7332 1st Qu.: 4.305 1st Qu.:0.0014362
## Median :  7.6616 Median :11.507 Median :0.0025485
## Mean   :  6.4396 Mean   :10.797 Mean   :0.0030832
## 3rd Qu.: 14.1183 3rd Qu.:16.573 3rd Qu.:0.0041187
## Max.   : 27.0018 Max.   :22.500 Max.   :0.0137057
```

Task c

```
print(f"Mean: {np.mean(df['T84.mean'].values)}")
```

```
## Mean: 6.439594405677285
```

```
print(f"Standard deviation: {np.std(df['T84.mean'].values)}")
```

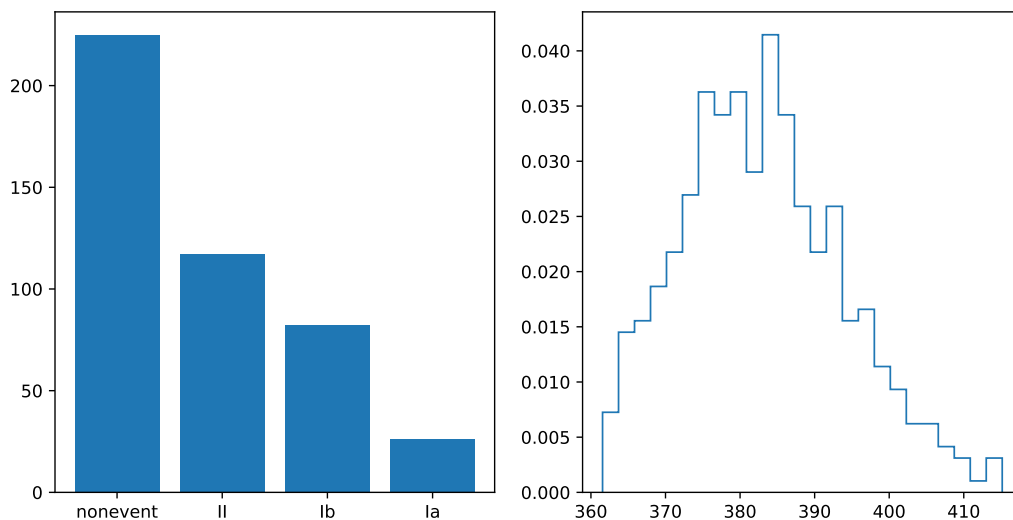
```
## Standard deviation: 9.816356767300478
```

```
# R
cat(mean(df$T84.mean), sd(df$T84.mean))
```

```
## 6.439594 9.827282
```

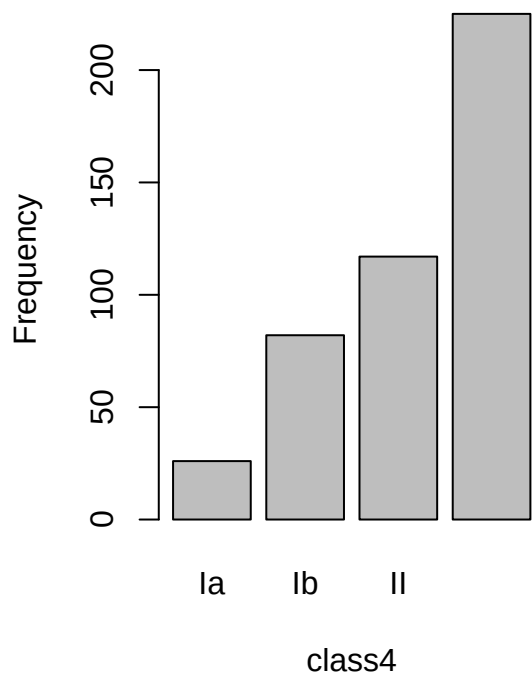
Task d

```
# Python
class_counts = df["class4"].value_counts()
fig, axes = plt.subplots(1,2,figsize=(10,5))
_ = axes[0].bar(class_counts.index, class_counts.values)
_ = axes[1].hist(df["C0242.mean"].values, bins=25, histtype="step", density=True)
plt.show()
```

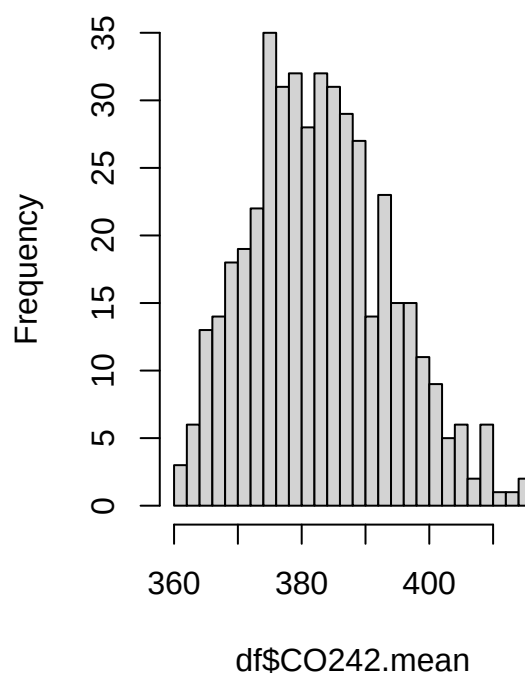


```
# R
par(mfrow = c(1, 2))
barplot(table(df$class4), ylab="Frequency", xlab="class4", main = "Bar plot of Class4")
hist(df$CO242.mean, breaks = 20, main = "Histogram for CO2 4.2m")
```

Bar plot of Class4



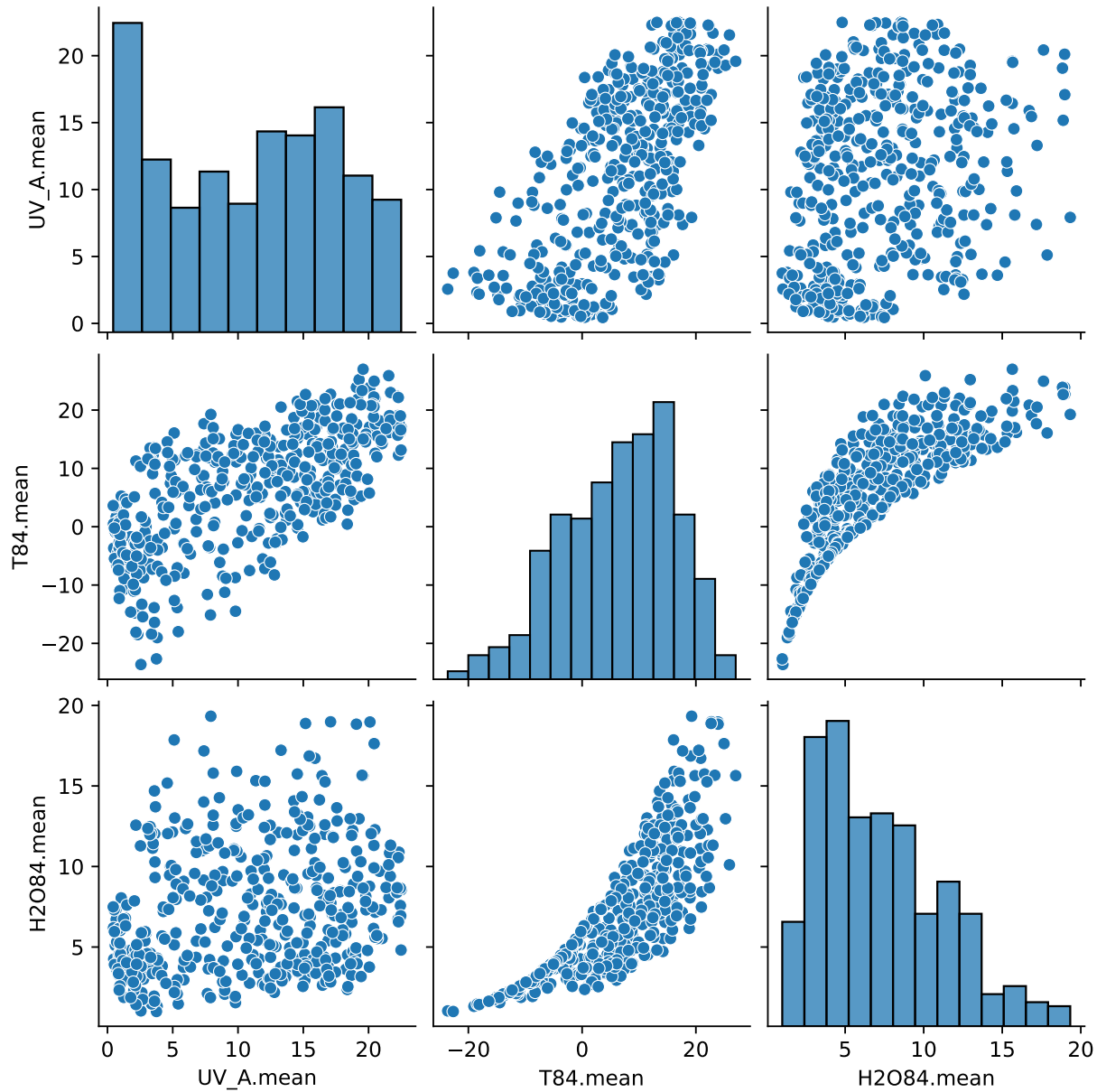
Histogram for CO2 4.2m



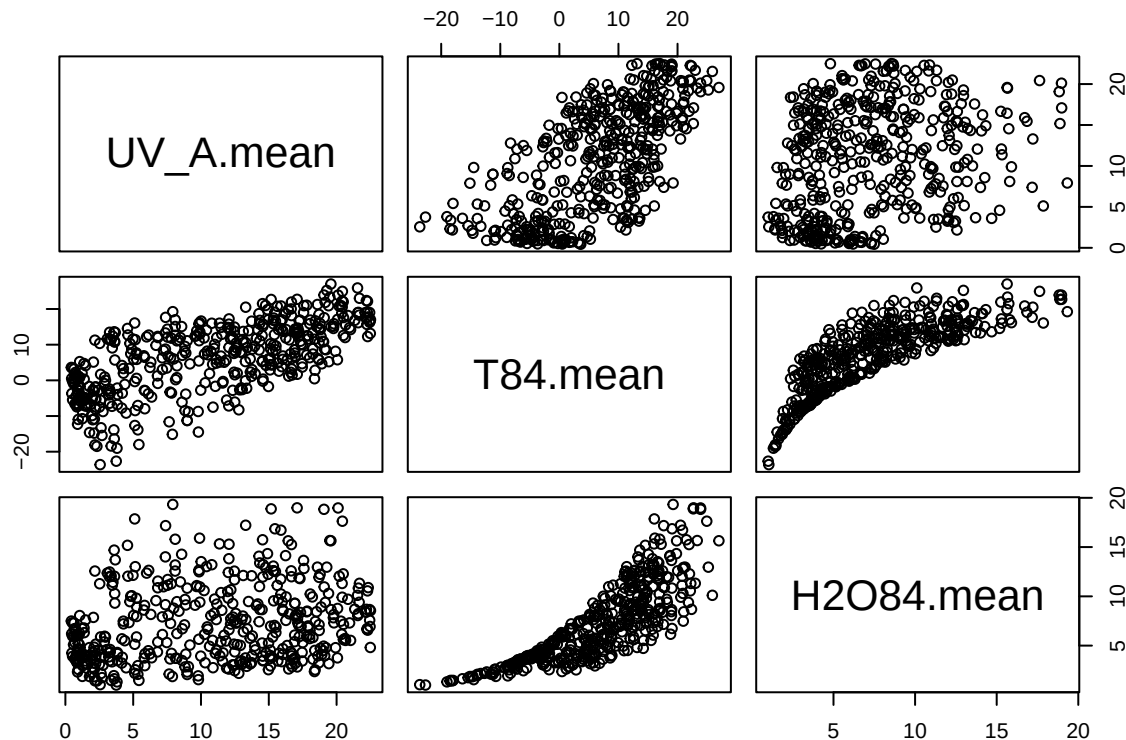
Task e

```
# Python
sns.pairplot(df[["UV_A.mean", "T84.mean", "H2084.mean"]])
```

```
## <seaborn.axisgrid.PairGrid object at 0x772000887fb0>
```



```
# R  
pairs(df[,c("UV_A.mean", "T84.mean", "H2O84.mean")])
```



Task f

Creating a dummy submission using R

```
train = read.csv("npf_train.csv")
test = read.csv("npf_test.csv")
# count class occurrences in train
class_table = table(train$class4)
# pick the most common class
most_probable = names(which.max(class_table))
dummy_probability = sum(train$class4 != "nonevent") / length(train$class4)
yhat = rep(most_probable, length(test$id))
p = rep(dummy_probability, length(test$id))
# create submission.csv
write.csv(data.frame(id=test$id, class4=yhat, p=p), "./submission.csv", row.names = FALSE)
```

Using Python

```
# read the training data
train_df = pd.read_csv("npf_train.csv")
# find the most common class
most_probable = train_df["class4"].mode().values[0]
dummy_probability = np.sum(train_df["class4"] != "nonevent") / len(train_df.index)

# create submission.csv
test_df = pd.read_csv("npf_test.csv")[["id"]]
test_df.loc[:, "class4"] = most_probable
test_df.loc[:, "p"] = dummy_probability
test_df.to_csv("submission.csv", index=False)
```

The score for a dummy model should be around 0.36269 (depending on numerical accuracy).

Grading

Points: max. 6 (Tasks a-f: 1 point each)

It is enough to report that the instructed commands were run in Tasks a-b. The important thing is that the data set is preprocessed as instructed (in a manner done in the programming environment) and the requested analysis is performed.

Problem 2

Task a

The requested table is shown below.

Degree	Train	Validation	Test	TestTRVA	CV
0	18.460	32.342	22.101	21.620	27.212
1	4.089	7.128	8.876	9.935	9.634
2	0.219	0.294	0.246	0.214	0.352
3	0.217	0.283	0.290	0.275	0.561
4	0.119	0.625	0.969	0.224	2.028
5	0.097	0.573	4.895	1.039	5.812
6	0.008	3.417	213.297	0.881	2.979
7	0.005	6.863	1261.988	0.272	213.171
8	0.002	401.652	154266.871	11.224	7820.298

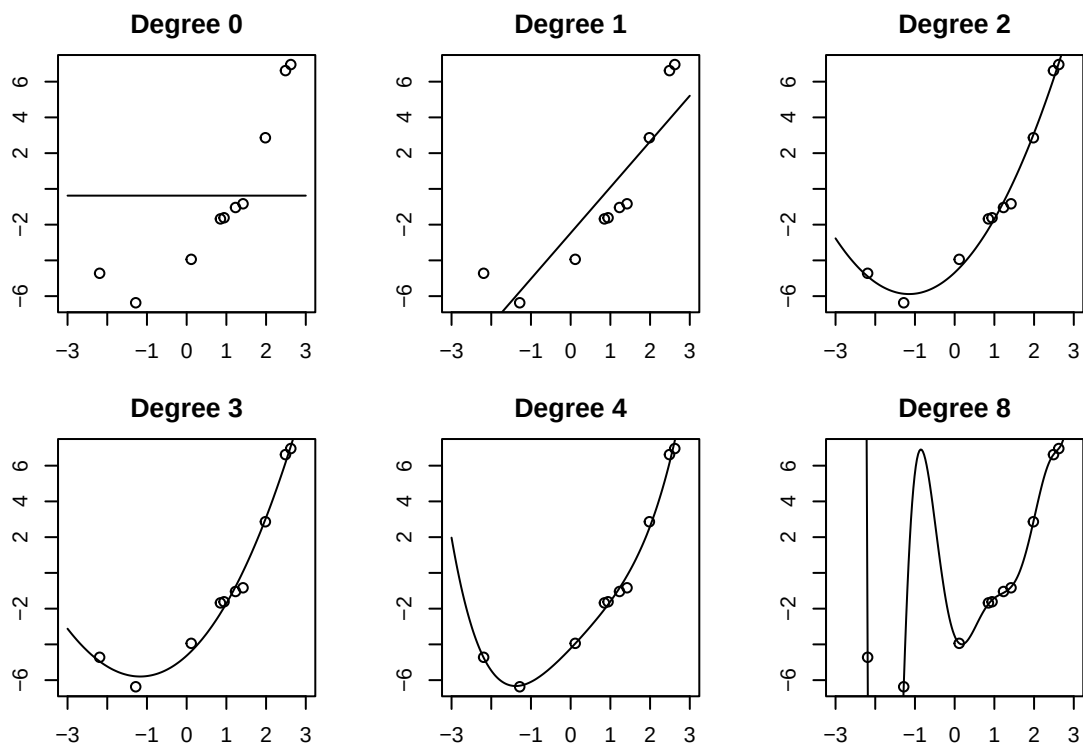
We choose the polynomial order according to the **CV** loss, which is minimized for the 2nd degree polynomial. We would train the “final” model by using the combined training and validation set.

Some things to notice:

- **Train** decreases for larger polynomial degrees.
- **Test** is smallest for the degree 2 polynomial, which is consistent with the fact that data is created by a degree 2 polynomial plus noise (which was not known to you).
- **TestTRVA** is lower than **Test**, as expected, since we train the regressor on more data.
- **Validation** is smallest for the degree 3 polynomial, due to the quite small validation set.
 - We would expect that $E[\text{Train}] < E[\text{Validation}] < E[\text{Test}]$. However, this expectation does not always hold, due to variance in the estimators (e.g., the validation set has only 10 items).
- **CV** is smallest for the degree 2 polynomial, which is the degree we should choose if we would trust the CV result. Also note that we have chosen *not* to shuffle the data prior to performing cross validation. Hence, you may see much lower values for **CV** if you performed shuffling. Shuffling is also perfectly valid, and might even be preferable in some cases. The overall trend in **CV** should be similar to that in **Test** regardless of shuffling.

Task b

The plots for the polynomials are shown below.



Task c

The requested table is shown below. As an additional model we selected a regression tree (RT).

	Train	Test	CV
Dummy	3.089	2.998	3.090
OLS	1.378	1.484	1.381
RF	0.571	1.448	0.686
SVM	0.978	1.530	1.041
RT	1.548	1.771	1.496

1. The best regressor is RF because it has the smallest CV and test losses (even if it overfits to the training data). The simple baseline OLS is also surprisingly good on test data.
2. **Train** errors are smaller than **Test** errors. **CV** and **Test** errors are roughly the same, as expected. An exception is the dummy model, since it is so simple and the train and test sets are drawn from the same distribution.
3. The regressors can be improved in several ways, such as:
 - Feature selection. Try to use only a subset of the features which might lead to a better loss on the training data.
 - Parameter tuning. The regression models often include parameters that can be tuned, I could use the cross-validation loss to find variants of the models that would perform better.

Task c (Python)

```
##      train  test   cv
## dummy 3.089 2.998 3.086
## OLS   1.396 1.485 1.459
## SVR   3.084 2.997 3.087
```

```
## RF      0.532  1.474  1.422
## LASSO   1.434  1.509  1.485
```

Bonus observation: The SVR results are surprisingly bad (comparable to the dummy model). This is because we haven't scaled the data. Some implementations of SVM:s automatically scale the data (such as the one in the `e1071` package for R). Lets try the SVR from `sklearn` again, but this time preprocessing the data to have zero mean and unit variance:

```
##      train  test    cv
## SVR  1.418  1.626  1.707
```

As we can see, it is often a good idea to scale the variables before doing machine learning, since some models are sensitive to the scale of the variables. (This observation does not affect the grading)

Grading

Points: max. 8 (a: 3, b: 2, c: 3)

The answer should get full points if the asked for numbers from the table have been produced, as well as figures requested, and there additionally an answer to direct questions asked.

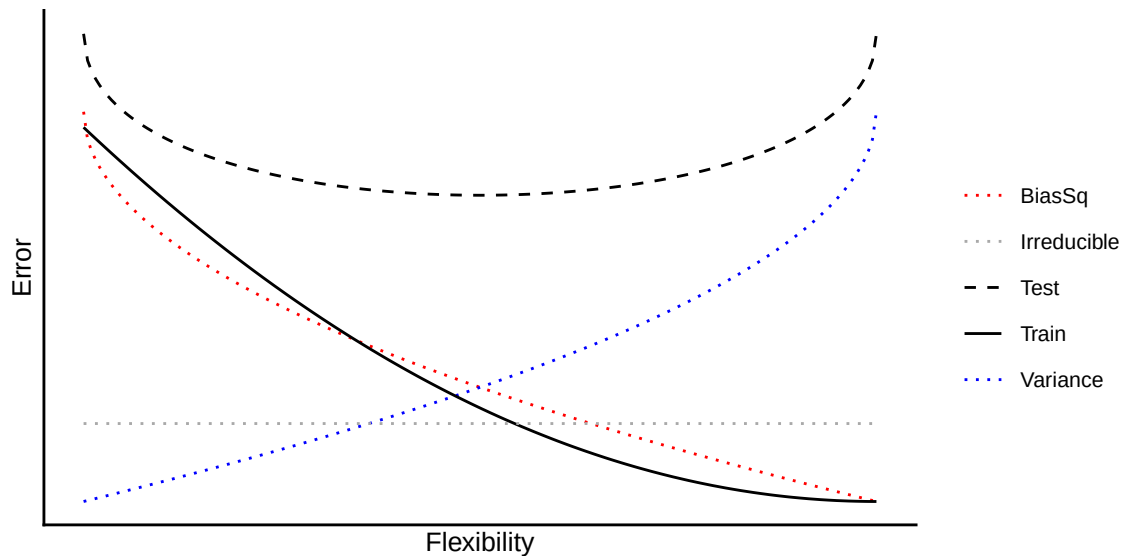
Problem 3

Task a

The curve shapes are explained as follows:

- The training error is usually smaller than the test error for any flexibility. It tends to decrease as model flexibility grows, going close to zero for very flexible models.
- The test error is large for small flexibility (under-fitting) and large flexibility (over-fitting), having the minimum in between. For regressors with MSE loss, the test error is the sum of the irreducible error, squared bias, and variance.
- The irreducible (or Bayes) error is constant, because it does not depend on the model. It gives a lower bound for the generalization error of any model for a particular task.
- The squared bias is large for inflexible models and small for flexible models. The opposite is true for variance.

A sketch of the curves is shown in the figure below (low flexibility on the left and large flexibility on the right).

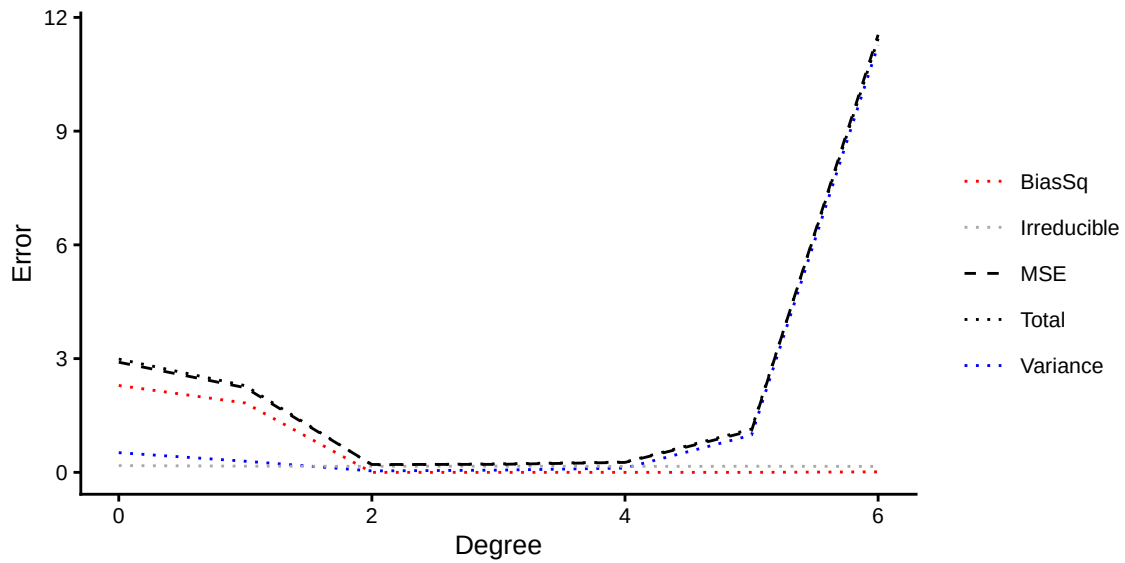


Task b

The requested table is shown below.

Degree	Irreducible	BiasSq	Variance	Total	MSE
0	0.1785	2.2916	0.5189	2.9889	2.9048
1	0.1639	1.8285	0.2933	2.2857	2.2332
2	0.1601	0.0000	0.0378	0.1979	0.2073
3	0.1641	0.0000	0.0587	0.2228	0.2115
4	0.1570	0.0001	0.1134	0.2706	0.2622
5	0.1611	0.0004	0.9815	1.1431	1.0997
6	0.1565	0.0125	11.2698	11.4387	11.5403

We also plot the terms from the table:



The bias-variance decomposition behaves as expected.

- The irreducible error is roughly constant (it does not depend on the regression model used),
- The squared bias decreases as flexibility (here polynomial degree) increases.
- The variance increases as flexibility (here polynomial degree) increases.
- We can verify Eq. (2.7) of James et al. by observing that the columns **Total** and **MSE** are roughly equal.

The small inconsistencies are due to the fact that we estimated the components by sampling 1000 datasets. The inconsistencies should be smaller if we would sample more datasets.

Grading

Points: max. 6 (a: 2, b: 4)

Problem 4

Tasks a

Since all test data points are drawn from the same population, we have $E \left[(\bar{y}_1 - \hat{\beta}^T \bar{x}_1)^2 \right] = \dots = E \left[(\bar{y}_m - \hat{\beta}^T \bar{x}_m)^2 \right]$. It follows that

$$E[L_{test}] = E \left[\frac{1}{m} \sum_{i=1}^m (\bar{y}_i - \hat{\beta}^T \bar{x}_i)^2 \right] = \frac{1}{m} \sum_{i=1}^m E \left[(\bar{y}_i - \hat{\beta}^T \bar{x}_i)^2 \right] = E \left[(\bar{y}_1 - \hat{\beta}^T \bar{x}_1)^2 \right],$$

where we have used the linearity of the expectation.

This proves the claim of the Task a.

Task b

The generalisation error for OLS regression is defined as the expected squared loss for a new data point not used in training. Therefore, $L = E \left[(\bar{y}_1 - \hat{\beta}^T \bar{x}_1)^2 \right] = E[L_{test}]$ is the generalisation error by definition. Since the expectation equals the estimated quantity the estimator is unbiased.

Notice that, as discussed in the problem statement, there are confusingly at least two different ways to define the expectation E and the generalisation error: (i) take the training data to be fixed and sample the test data, and (ii) sample over both training and test data. See Bengio et al.¹, Section 2.1, and Nadeau et al.² for discussion. Tasks a and b would work with either definition; in Task c we use the second definition.

Task c

It follows from Task a above that $E[L'_{test}] = E[L_{test}]$ as well, where

$$L'_{test} = \frac{1}{n} \sum_{i=1}^n (\bar{y}'_i - \hat{\beta}^T \bar{x}'_i)^2,$$

and $(\bar{x}'_1, \bar{y}'_1), \dots, (\bar{x}'_n, \bar{y}'_n)$ have been drawn randomly from the same population. In other words, the value of m does not really matter here.

We can use this observation prove our “theorem”.

Theorem: $E[L_{train}] \leq E[L_{test}]$.

Proof: By Task a and the observation above, the claim is equivalent to $E[L_{train}] \leq E[L'_{test}]$. Recall that ordinary least squares (OLS) linear regression finds $\hat{\beta}$ such that the loss given by L_{train} is minimised, after which

$$L_{train} = \min_{\beta} \left(\frac{1}{n} \sum_{i=1}^n (y_i - \beta^T x_i)^2 \right).$$

The following equation obviously holds for any $\hat{\beta}$ (random variable or not), because the expression inside the expectation is always non-negative:

$$E \left[\frac{1}{n} \sum_{i=1}^n (\bar{y}'_i - \hat{\beta}^T \bar{x}'_i)^2 - \min_{\beta} \left(\frac{1}{n} \sum_{i=1}^n (\bar{y}'_i - \beta^T \bar{x}'_i)^2 \right) \right] \geq 0$$

By linearity of expectation we can rewrite the above as;

$$E \left[\frac{1}{n} \sum_{i=1}^n (\bar{y}'_i - \hat{\beta}^T \bar{x}'_i)^2 \right] - E \left[\min_{\beta} \left(\frac{1}{n} \sum_{i=1}^n (\bar{y}'_i - \beta^T \bar{x}'_i)^2 \right) \right] \geq 0$$

¹Bengio, Y., Grandvalet, Y., 2004. No unbiased estimator of the variance of K-fold cross-validation. J. Mach. Learn. Res. 5, 1089-1105. <https://www.jmlr.org/papers/v5/grandvalet04a.html>

²Nadeau, C., Bengio, Y., 2003. Inference for the Generalization Error. Mach. Learn. 52, 239-281. <https://doi.org/10.1023/A:1024068626366>

The latter term is $E[L_{train}]$, as explained earlier, and the first term is $E[L'_{test}] = E[L_{test}]$, or

$$E[L_{test}] - E[L_{train}] \geq 0,$$

which proves the theorem.

Task d

Theorem 2 shows that the expected loss on the training data cannot be larger than the expected loss on the test data. In machine learning, we usually want to estimate the *generalization loss* L .

Consider the case where we use the loss on training data L_{train} as an estimator of the generalisation loss L . The difference between the expectation of the estimator and the ground truth is called the *bias* of the estimator, which can here be written as $\text{bias} = E[L_{train}] - L = E[L_{train}] - E[L_{test}]$. For finite training data (n finite) and OLS linear regression, this bias is always negative, which means that we tend to underestimate the generalisation loss and that OLS linear regression tends to overfit to the training data.

Grading

You should give full or almost full points *if* the idea is correct, even though the proofs would be “shaky”. However, please point out any ambiguities in your review.

Points: max. 6 (a: 2, b: 1, c: 2, d: 1)

Problem 5

The dataset in this problem is the famous [Anscombe's quartet](#).

Task a

We use R's built-in `summary` function to obtain the required values, shown below.

We can reject the null hypothesis (at level $\alpha = 0.05$) that the slope is zero, either by observing the coefficient for `x` has a p-value less than α , or equivalently, by observing that zero is not in the 95% confidence interval for `x`, given by `Estimate  $\pm$  2 * Std. Error`. The small p-value gives some indication that an increase in `x` is associated with an increase or decrease in `y`, but it is not a “safe conclusion” because we are not sure if assumptions of linear regression hold for a given dataset.

```
## D1
## Call:
## lm(formula = y ~ x, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.921 -0.456 -0.041  0.709  1.839
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      3.00      1.12     2.7   0.026
## x                0.50      0.12     4.2   0.002
##
## Residual standard error: 1.2 on 9 degrees of freedom
## Multiple R-squared:  0.67,    Adjusted R-squared:  0.63
## F-statistic: 18 on 1 and 9 DF,  p-value: 0.0022
##
## D2
## Call:
## lm(formula = y ~ x, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.90  -0.76   0.13   0.95   1.27
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      3.00      1.13     2.7   0.026
## x                0.50      0.12     4.2   0.002
##
## Residual standard error: 1.2 on 9 degrees of freedom
## Multiple R-squared:  0.67,    Adjusted R-squared:  0.63
## F-statistic: 18 on 1 and 9 DF,  p-value: 0.0022
##
## D3
## Call:
## lm(formula = y ~ x, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.16  -0.61  -0.23   0.15   3.24
##
```

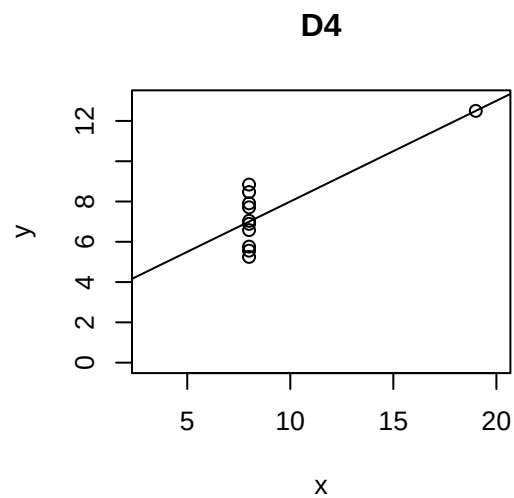
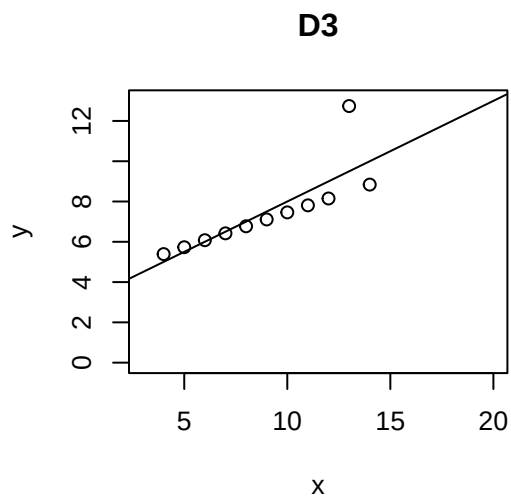
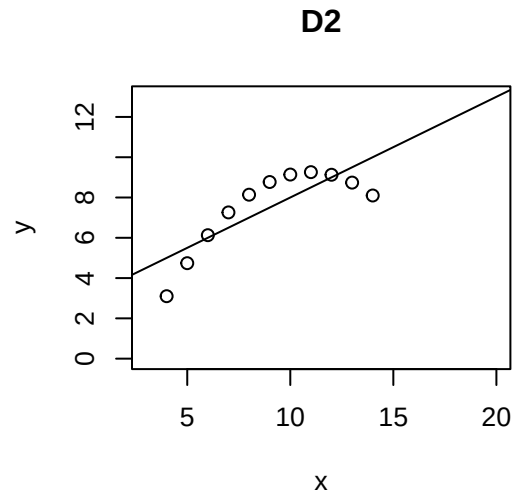
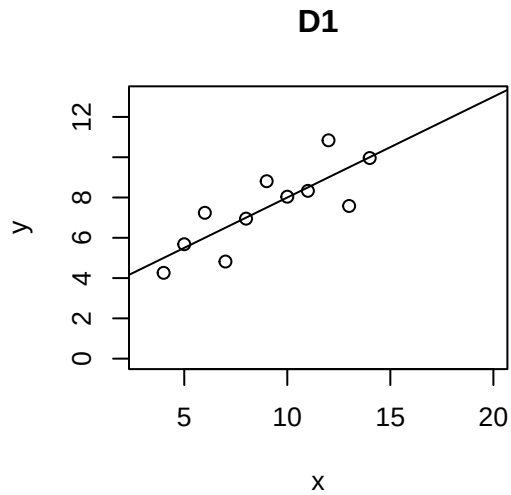
```
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)      3.00      1.12     2.7   0.026
## x                0.50      0.12     4.2   0.002
##
## Residual standard error: 1.2 on 9 degrees of freedom
## Multiple R-squared:  0.67,    Adjusted R-squared:  0.63
## F-statistic: 18 on 1 and 9 DF,  p-value: 0.0022
##
## D4
## Call:
## lm(formula = y ~ x, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.75  -0.83   0.00   0.81   1.84
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)      3.00      1.12     2.7   0.026
## x                0.50      0.12     4.2   0.002
##
## Residual standard error: 1.2 on 9 degrees of freedom
## Multiple R-squared:  0.67,    Adjusted R-squared:  0.63
## F-statistic: 18 on 1 and 9 DF,  p-value: 0.0022
```

Task b

The data and their regression lines are shown below.

We notice that the regression lines are the same, although the data sets look very different. The [data sets](#) also have the same average x, average y, standard deviation of x, standard deviation of y, correlation between x and y, and R squared value.

The conclusion here is that you should always plot the data and the model. Don't blindly trust loss values or p-values (or even averages).



Task c

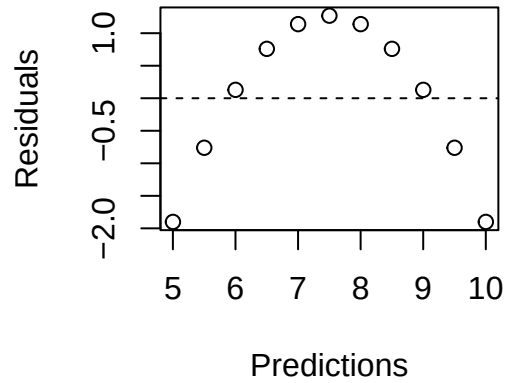
James et al. Sect. 3.3.3 lists 6 potential problems for linear regression:

1. Non-linearity of the response-predictor relationship
2. Correlation of error terms
3. Non-constant variance of error terms
4. Outliers
5. High-leverage points
6. Collinearity

Out of these problems, the following apply to each data set:

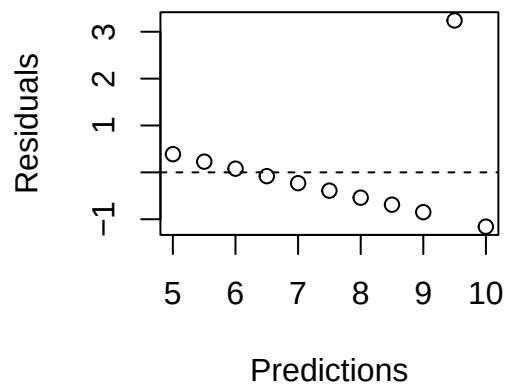
- D2: y is a *non-linear* function of x , as we can see from the scatterplot above, or from a residual plot:

D2



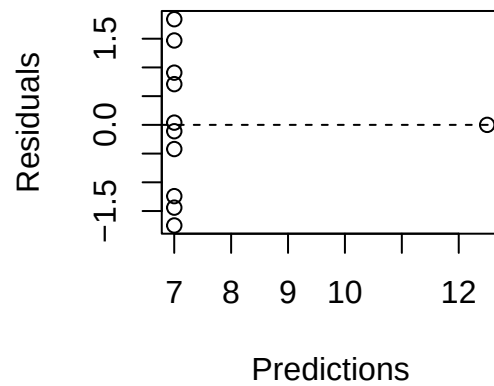
- D3: There is an *outlier* point evident from the scatterplot above, or from the residual plot below. Robust regression would ignore the outlier.

D3



- D4: There is a *high-leverage* point (outlier in x) evident from the scatterplot above, or from the residual plot below. If we delete the point, the estimates change a lot.

D4



Grading

Points: max. 6 (a: 2, b: 2, c: 2)

Problem 6

Task a

We follow here ISLR_v2 lab section Section 5.3.4. First, we make a function that fits a linear regression model to the data for a given subset of data points and outputs the regression coefficients.

```
## Use data from the previous problem
## From Sect. 5.3 of ISLR_v2:
boot.fn <- function(d, index)
  coef(lm(y ~ x, data=d, subset=index))
boot.fn(data[[2]], 1:nrow(data[[2]]))
```

```
## (Intercept)          x
##    3.000909    0.500000
```

Bootstrap samples new data set by replacement and fits a linear model. R function `boot` does this automatically, even though implementing it all by yourself would not be too complicated.

```
library(boot)
set.seed(42)
boot(data[[2]], boot.fn, 1000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = data[[2]], statistic = boot.fn, R = 1000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1*  3.000909  0.10796624   1.4875855
## t2*  0.500000  0.003024308   0.1602044
```

For dataset 2 the standard deviation for intercept (1.48 vs. 1.12) and slope (0.16 vs 0.12) are larger than in the previous problem. The difference is due to the fact that the t-test used in the previous problem involves several assumptions (normality, homoscedasticity, etc.) which may be violated in dataset 2. Bootstrap makes fewer distributional assumptions and, hence, in this case probably the bootstrap estimate is more trustworthy and robust. Notice that you should be careful in interpreting the regression coefficients for dataset 2 because our modelling assumptions seem to be wrong: the dataset looks like a curved line and it is doesn't seem that the underlying model is a straight line plus uncorrelated homoscedastic noise (our modelling assumption).

Taks b

The algorithm creates many bootstrap datasets from dataset 2. To create a bootstrap dataset, it draws n (x, y) -samples with replacement from dataset 2. For each bootstrap dataset, it fits a linear model and obtains the corresponding values for intercept and slope. The bootstrap standard deviations for the intercept and slope are the standard deviations of the intercept and slope, respectively, of the lines fitted to the bootstrapped datasets.

Task c

A bootstrap sample consists of n items drawn out of $\{1, \dots, n\}$ with replacement.

Let's look at one item $j \in \{1, \dots, n\}$. The item j is not in a draw with a probability of $1 - 1/n$. Since draws are independent the item j is not in any of the n draws with a probability $(1 - 1/n)^n$. At the limit $n \rightarrow \infty$ the

item j is therefore missing from the bootstrap sample with a probability of $\lim_{n \rightarrow \infty} (1 - 1/n)^n = 1/e \approx 0.368$. Or equivalently, the item j is in the bootstrap sample with a probability of $1 - 1/e \approx 0.632$.

Therefore roughly a third of the items is missing from a bootstrap sample!

Grading

Points: max. 6 (a: 2, b: 2, c: 2)

Problem 7

Give full points if the answer contains some sentences that are about the topic of the question.

Grading

Points: max. 2